

## 2021년 10월 4일 변경점

1. multichannel data를 읽을 수 있도록 함
2. 데이터를 불러올 때 각 row의 observation 수를 256으로 고정했던 것을 바꿔서 파일의 row당 observation 수를 따라가도록 함
3. 그림 그리는 method를 대부분 삭제하고 channel object에서 데이터만 불러와서 matlab 기본 코드로 직접 plotting하는 쪽으로 변경(plot의 스케일, 색상 등을 더 자유롭게 조절할 수 있도록 하기 위해)
4. bandpass filtering 시 preview를 볼 수 있도록 함
5. figure 4f의 phase-space plot에서 normalizing할 때 mean을 빼는 것에서 min으로 나누는 것으로 바꿈
6. channel object 관련 코드가 channel.m 이 아니라 channel\_from\_data.m에 저장됨

## 파일 목록

1. channel\_from\_data.m : 데이터 및 변수의 효율적 관리를 위해 만든 channel class 정의 파일. CyborgBrainOrg.m의 내용이 리팩토링되어 들어가 있음
2. channelsFromMultiChannel.m: multichannel 불러오기 관련 코드
3. longtermAnalyzer.m : 여러 organoid, channel, month 데이터를 통합 관리하고 fwhm을 계산하고 month 별로 묶어 plotting하기 위해 만든 longtermAnalyzer class 정의 파일(ex. figure 2.o에서 사용)
4. summary\_examples.mlx: 9월 미팅 때 사용한 설명 및 예시 파일
5. summary\_examples\_multichannel.mlx: 본 파일
6. CyborgBrainOrg.m : 원본 reference code를 주어진 데이터에서 돌아가도록 수정한 파일(9월 이후 수정하지 않음, single channel, 256 observation per row, MEA data.xlsx 와 똑같은 포맷에서만 작동)

## 요약

### 1. Reference code(CyborgBrainOrg.m)에 포함된 내용

1. spike detection algorithm
2. cluster average waveform with confidence band
3. raster plot
4. colored raster plot
5. PCA
6. kmeans
7. calculation of discrete derivative(dV/dt)
8. ISI(interspike interval) plot

### 2. Reference code에 포함되지 않아서 제가 추가한 내용

1. Baw data file을 불러와서 정리하여 array로 바꾸는 함수
2. Bandpass filter 적용을 위한 uniform resampling과 [300hz, 3000hz] bandpass filter 적용
3. Bandpass filter 적용을 통한 theta wave 획득 및 Hilbert transform을 통한 theta phase 계산
4. circstat 패키지를 이용한 theta phase 분포 plot 및 phase lock test
5. phase-space plot
6. FWHm(full width half minimum) 계산 및 month 단위로 합쳐서 bar plot으로 비교

### 3. Reference code를 변경한 내용

1. 기존 코드는 sampling frequency 30000에 맞춰서 모든 코드가 구성되어 있어서, sampling frequency가 다른 데이터에 적용하면 제대로 작동하지 않았음(ms <-> timestamp 간 변환 관련) -> 데이터의 sampling frequency를 자동으로 계산하고, 계산한 값에 맞추어 다른 알고리즘이 동작하도록 수정
2. PCA 수행 시 standardization 시행
3. k-means 수행 시 seed number 사용
4. 여러 채널의 데이터를 효율적으로 관리하고 관련 데이터와 변수를 한 곳에 모아 놓기 위해 class 제작: channel이라는 object에 데이터 파일 하나의 raw data, filtered data, PCA 및 kmeans 결과 등이 저장됨
5. plotting은 channel object에서 필요한 데이터를 불러온 후 matlab 기본 함수를 통해 그리는 것을 원칙으로 함. raster plot의 경우에만 따로 함수를 두었음

### 4. 필요한 애드온 목록(설치하지 않으면 코드가 작동하지 않음)

1. Statistics and Machine Learning Toolbox
2. Signal Processing Toolbox
3. Bioinformatics Toolbox
4. Circular Statistics(by Philipp Berens)

### 6. 구현 못한 plot

1. figure 4.e figure 2.o에서 FWHm만 signal power로 바꾸면 됨. signal power를 matlab에서 계산하는 방법을 아직 찾지 못함
2. figure 4.h는 figure 2.o에서 FWHm만 spikes count per neuron으로 바꾸면 됨.
3. figure 4.i

## channel class 설명 및 논문의 데이터 처리 재현과 코멘트

### 0. channel object 생성 및 데이터 처리

0.1. (class) channelsFromMultiChannel: 여러 채널이 존재하는 데이터 파일 불러오기

## parameter 1개

- **filename**: 같은 디렉토리일 경우 파일이름만, 다른 디렉토리일 경우 전체 경로 포함 입력. "1\_singleunit wave.xlsx" 파일과 동일한 포맷이어야 함

85mb짜리 "1\_singleunit wave.xlsx"파일을 로드하는 데에 2.4ghz i7 쿼드코어에서 약 90초 소요

```
file = channelsFromMultiChannel("1_singleunit wave.xlsx");
```

## 0.2. (method of channelsFromMultiChannel) channel: channel object 생성 및 resampling 수행

### parameter 3개

- **organoid**: organoid 번호 (관리를 위한 번호, 제한 없음)
- **channel**: channel 번호. 위에서 불러온 파일 내에 존재하는 채널 번호여야 함.
- **month**: month. 나중에 month별로 종합할 때 사용됨 (관리를 위한 번호, 제한 없음)

object를 생성할 때 데이터를 불러오고 bandpass filter 적용을 위한 resampling을 수행함.

```
% organoid, channel, month  
o1_ch8_m1 = file.channel(1, 8, 1); %8번 채널 데이터를 불러옴
```

### 0.2.1 (class) channel: single channel data의 channel object 생성, 데이터 불러오기 및 resampling 수행

#### parameter 4개

- **filename**: 같은 디렉토리일 경우 파일이름만, 다른 디렉토리일 경우 전체 경로 포함 입력. MEA data.xlsx 파일의 sheet 1 과 동일한 포맷이어야 함
- **organoid**: organoid 번호 (관리를 위한 번호, 제한 없음)
- **channel**: channel 번호 (관리를 위한 번호, 제한 없음)
- **month**: month. 나중에 month별로 종합할 때 사용됨 (관리를 위한 번호, 제한 없음)

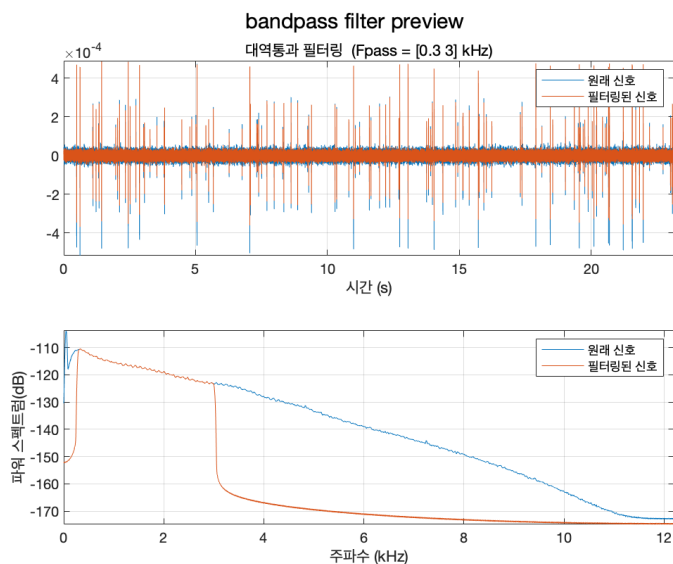
채널 1개 데이터만 존재하는 파일(MEA data.xlsx와 같은 포맷)일 경우, channelsFromMultiChannel를 거치지 않고 바로 channel object 생성 가능

```
% organoid, channel, month  
o2_ch1_m1 = channel("MEA data.xlsx", 1, 1, 1);
```

### 0.3.1 bandpass filter preview

channel object에서 raw 데이터를 불러와서 bandpass filter 적용시 어떤 결과가 나오는지 그래프로 확인

```
%apply bandpass filter 300-3000hz  
figure  
bandpass(o1_ch8_m1.raw, [300,3000], o1_ch8_m1.sf)  
sgtitle("bandpass filter preview")
```



### 0.3.1 (method) bandPass

#### bandpass filter 적용

##### 파라미터 1개

- **passBand** : [숫자, 숫자] 포맷으로, hz 단위로 입력

bandpass filter 실제로 적용하여 channel object 내에 저장

```
o1_ch8_m1.bandPass([300,3000]); %apply bandpass filter 300-3000hz
```

### 0.4. (method) detectSpikes

#### spike detection 수행

### parameter 3개

- **thres**: spike 분류 기준. 5를 입력하면 평균+5표준편차 밖의 값을 spike로 분류 ((reference code에서는 5)
- **preTime**: spike waveform의 spike 앞쪽 너비. milisecond 단위 (reference code에서는 2)
- **postTime**: spike waveform의 spike 뒷쪽 너비. milisecond 단위 (reference code에서는 2)

주의: 코드 내 threshold에 minus가 붙어 있음, 즉 이 알고리즘은 local minimum을 찾는 알고리즘. 레퍼런스 코드에서 이렇게 구현되어 있었음.

```
%thres, preTime, postTime
o1_ch8_m1.detectSpikes(5,2,2);
```

number of spikes found : 113

### 0.5. (method) getPCScores

#### PCA 수행

주의

- PCA는 각 축의 분산을 최대한 보존하는 principal component를 찾는 알고리즘이므로, 특정 waveform의 scale이 지나치게 크다면 분산에 많이 기여하게 되어 그 waveform을 principal component로 잘못 인식할 수 있음. 일반적으로 PCA를 비롯한 차원 축소 알고리즘들이 비슷한 문제를 가지고 있기 때문에, 알고리즘 수행 전에 각 데이터포인트의 scale을 통일하는 scaling을 필수적으로 수행함
- Scaling 방법은 평균을 빼고 표준편차로 나누는 standard scaling, min을 빼고 (max-min)으로 나누는 0-1 scaling 등이 있음
- 물론 데이터가 scale이 원래 일정하다면 굳이 standardization을 수행할 필요가 없음. Reference code에서는 standard scaling 코드를 써 놓고 주석 처리해 놓았음. 이 코드에서는 주석 처리를 해제하고 standard scaling을 수행하도록 하였음
- PCA 수행 후 explained variance plot을 꼭 확인해야 함. 첫 두 PC가 설명하는 분산의 양이 충분히 커야(보통 70% 이상) 2차원만 남기고 나머지 차원을 버리는 것이 정당화됨
- 경고 메시지 "X의 열은 기계 정밀도 내에서 1차 종속입니다"는 spike waveform의 시작과 끝 부분이 평탄해서 비슷한 값을 가지기 때문에 발생. 어차피 PCA를 통해서 차원 축소를 할 것이기 때문에 문제되지 않음

```
o1_ch8_m1.getPCScores();
```

경고: X의 열은 기계 정밀도 범위 내에서 1차 종속입니다.  
처음 51개 성분만 사용하여 TSQUARED가 계산됩니다.

### 0.6. (method) getKmeansClusters

#### K-means clustering 수행

#### parameter 2개

- **clusternum**: cluster 개수(reference code에선 3)
- **seednum**: 알고리즘의 초기값에 영향을 미치는 random number seed(reference code에 없음)

주의

- k-means clustering은 해를 해석적으로 구하지 못하고 EM algorithm이라는 iterative algorithm을 사용하는데, 초기값에 큰 영향을 받음. 초기값이란 각 점의 맨 처음 cluster 배정을 의미하며, 랜덤으로 할당함. 일반적으로 이러한 EM algorithm 기반 clustering을 수행할 때는 여러 seed number를 사용하여 알고리즘을 여러 번 돌려봄.
- k-means clustering에서 cluster 개수(clusternum)는 자동으로 최적값이 정해지지 않으며, 정답이 없는 비지도학습이기에 최적값을 알 수도 없음. 일반적으로 사용자가 여러 clusternum을 시도해 보며 적절한 값을 찾아야 함.
- 결론적으로, 여러 clusternum과 seednum의 조합을 시도하면서 plot을 보고 사용자 눈에 적절한 결과가 나올때까지 진행해야 함.

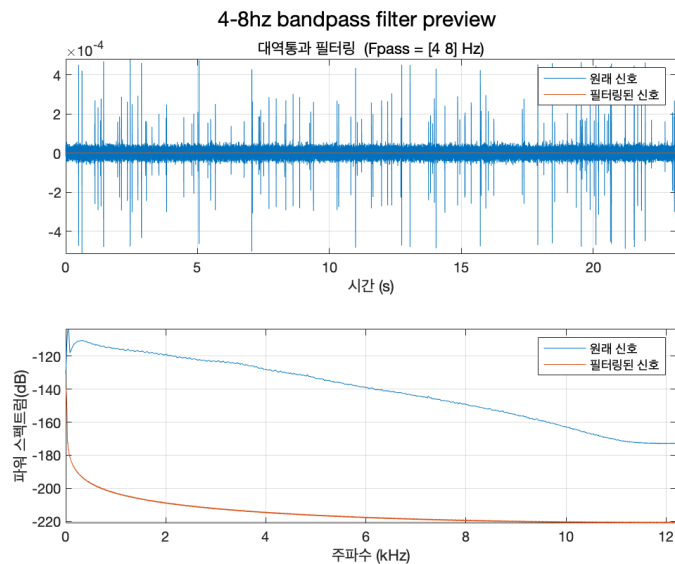
```
%clusternum, seednum
o1_ch8_m1.getKmeansClusters(3,2212323231);
```

### 0.7.1 theta wave preview

channel object에서 raw 데이터를 불러와서 [4hz,8hz] bandpass filter 적용시 어떤 결과가 나오는지 그래프로 확인

계산에 약 10초 정도 걸림

```
%apply bandpass filter 4-8hz
figure
bandpass(o1_ch8_m1.raw, [4,8], o1_ch8_m1.sf)
sgtitle("4-8hz bandpass filter preview")
```



### 0.7.2. (method) getThetas

theta wave 및 theta angle 계산

parameter 없음

논문에 나온 대로 계산

- theta wave: raw trace에 4-8hz bandpass 적용
- theta angle: theta wave에 hilbert transform 적용 후 실수부와 허수부 사이의 angle을 계산

계산에 약 10초 정도 걸림

```
o1_ch8_m1.getThetas();
```

## 1. figure 2 plot 재현 및 코멘트

### Figure 2.f.

figure 2.b-d는 2.f.와 같은 내용.

raw trace, filtered trace, raster plot

figure 2.f. 그리는 데에 필요한 정보

- **channelobject.t** : time array
- **channelobject.raw**: raw trace array
- **channelobject.filtered**: filtered trace array

이후 matlab 기본 plot 함수 사용, raster plot만 **drawRaster** method 사용

```
figure

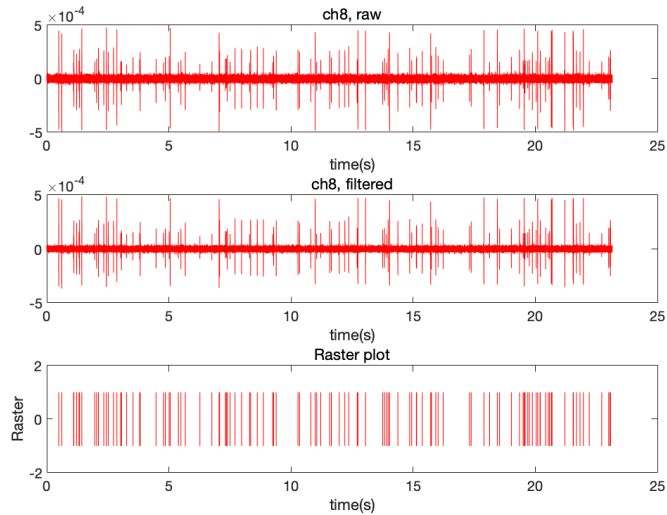
%raw trace plot
subplot(3,1,1)
pRaw = plot(o1_ch8_m1.t, o1_ch8_m1.raw); %matlab 기본 plot 함수 사용
pRaw.Color = 'red'; %색깔 조절 가능
title('ch8, raw'); %제목
ylim([-5,5]*1e-4); %ylim 조절 가능
xlim([0,25]); %xlim 조절 가능
xlabel("time(s)")

%filtered trace plot
subplot(3,1,2)
pFiltered = plot(o1_ch8_m1.t, o1_ch8_m1.filtered); %matlab 기본 plot 함수 사용
pFiltered.Color = 'red'; %색깔 조절 가능
title('ch8, filtered');
ylim([-5,5]*1e-4); %ylim 조절 가능
xlim([0,25]); %xlim 조절 가능
xlabel("time(s)")

%raster plot of spike locations
subplot(3,1,3) %제목
o1_ch8_m1.drawRaster('red'); %색깔 조절 가능
xlim([0,25]); %xlim 조절 가능

sgtitle("figure 2.f. raw trace, filtered trace, raster plot of detected spike locations")
```

figure 2.f. raw trace, filtered trace, raster plot of detected spike locations



## Figure 2.g.

figure 2.e와 같은 내용

filtered trace에서 찾은 spike의 위치를 raw trace와 filtered trace에서 zoom in

여기서는 예제로 첫번째 spike의 위치를 zoom in

figure 2.g. 그리는 데에 필요한 정보

- **channelobject.spikeTimestamps**: filtered trace에서 찾아낸 spike들의 위치(timestamp 단위)
- **channelobject.timestampsPrePeak**: spike waveform의 전반부 너비(timestamp 단위. reference code에선 60으로 계산됨)
- **channelobject.timestampsPostPeak**: spike waveform의 후반부 너비(timestamp 단위)(timestamp 단위. reference code에선 60으로 계산됨)

이후 matlab 기본 plot 함수 사용

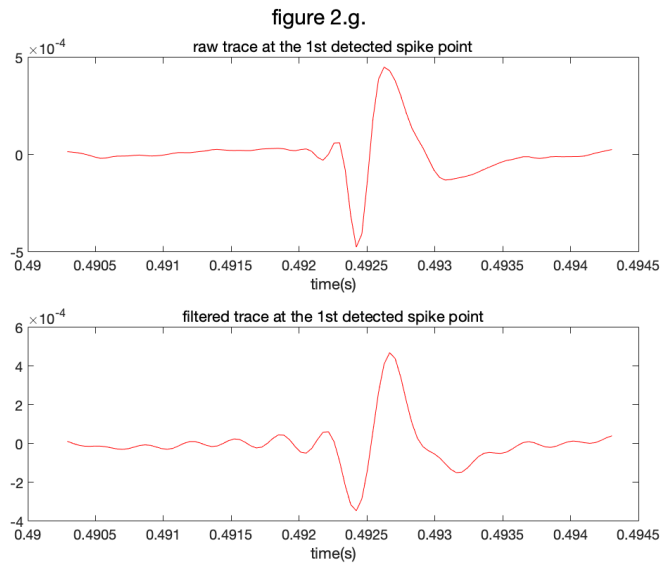
```
spikeTimestamps = o1_ch8_m1.spikeTimestamps; %spike 위치 가져오기
ts = spikeTimestamps(1); %예를 들어, 1번째 spike의 위치 가져오기
tsInterval = ts + (-o1_ch8_m1.timestampsPrePeak : o1_ch8_m1.timestampsPostPeak);
%spike를 중심으로 waveform의 t축 범위를 정함
```

figure

```
%raw trace zoom in
subplot(2,1,1)
pRaw = plot(o1_ch8_m1.t(tsInterval), o1_ch8_m1.raw(tsInterval));
pRaw.Color = 'red'; %색깔 조절 가능
title('raw trace at the 1st detected spike point');
xlabel('time(s)');

%filtered trace zoom in
subplot(2,1,2)
pFiltered = plot(o1_ch8_m1.t(tsInterval), o1_ch8_m1.filtered(tsInterval));
pFiltered.Color = 'red'; %색깔 조절 가능
title('filtered trace at the 1st detected spike point');
xlabel('time(s)');

sgtitle("figure 2.g.")
```



## Figure 2.h

PCA 및 kmeans 결과, 각 cluster별 average spike 모양

- explained variance plot: 첫 두 막대의 높이 합이 총분이 큰지 확인
- PCA/kmeans scatterplot: 결과를 보면서 여러 clusternum과 seednum 시도

explained variance bar plot 그리는 데에 필요한 정보

- `channelobject.explainedVar`

PCA plot 그리는 데에 필요한 정보

- `channelobject.PCScores`
- `channelobject.nClusters`: 클러스터 개수 정보
- `channelobject.clusters`: 각 spike의 cluster 할당 정보

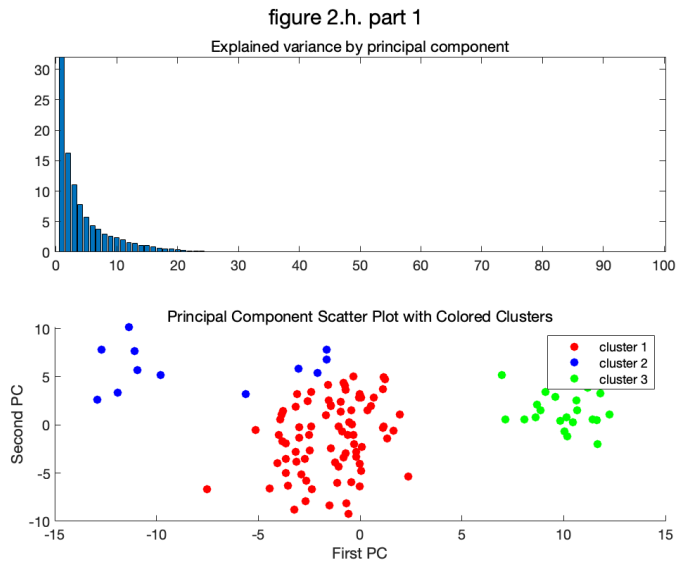
```
subplot(2,1,1)
bar(o1_ch8_m1.explainedVar); %matlab 기본 함수인 bar 함수 사용
title("Explained variance by principal component");

%PCA plot
subplot(2,1,2)
clusterColors = ["red", "blue", "green"]; %원하는 대로 설정
PC1 = o1_ch8_m1.PCScores(:,1);
PC2 = o1_ch8_m1.PCScores(:,2);

for c = 1 : o1_ch8_m1.nClusters
    %matlab 기본 함수인 scatter 사용
    scatter(PC1((o1_ch8_m1.clusters == c),:), PC2((o1_ch8_m1.clusters == c),:), clusterColors(c), 'filled')
    hold on;
end
hold off;

xlabel('First PC');
ylabel('Second PC');
title('Principal Component Scatter Plot with Colored Clusters');
legend("cluster " + string(1 : o1_ch8_m1.nClusters));

sgtitle("figure 2.h. part 1")
```



## Figure 2.h. part 2

mean spike of each cluster

cluster별 mean spike 정보 가져오는 method:

- `channelobject.drawClusterMeanSpikes(clusterNum)`

논문에서는 ylim 통일했음. 이 예제에서는 통일하지 않았음. ylim 옵션을 통해 쉽게 통일 가능

```
figure

%cluster 1
[meanSpikeWaveform_1, std_1, tRangeCentered, nSpikes] = o1_ch8_m1.getClusterMeanSpike(1);

subplot(3,1,1)
plot(tRangeCentered, meanSpikeWaveform_1 + 1*std_1, 'k--', 'LineWidth', 1, 'Color', "red"); %mean + 1sd 그리기
hold on;
plot(tRangeCentered, meanSpikeWaveform_1 - 1*std_1, 'k--', 'LineWidth', 1, 'Color', "red"); %mean - 1sd 그리기
hold on;

%curve 사이 색칠하기(오류 발생 가능)
t2 = [tRangeCentered, fliplr(tRangeCentered)];
inBetween = [meanSpikeWaveform_1 - 1*std_1, fliplr(meanSpikeWaveform_1 + 1*std_1)];
fill(t2, inBetween, "red");
hold on;

plot(tRangeCentered, meanSpikeWaveform_1, 'LineWidth', 2, 'Color', 'k'); %meanSpike 그리기

hold off;
title("mean spike of cluster 1")

%cluster 2
[meanSpikeWaveform_2, std_2, tRangeCentered_2, nSpikes_2] = o1_ch8_m1.getClusterMeanSpike(2);

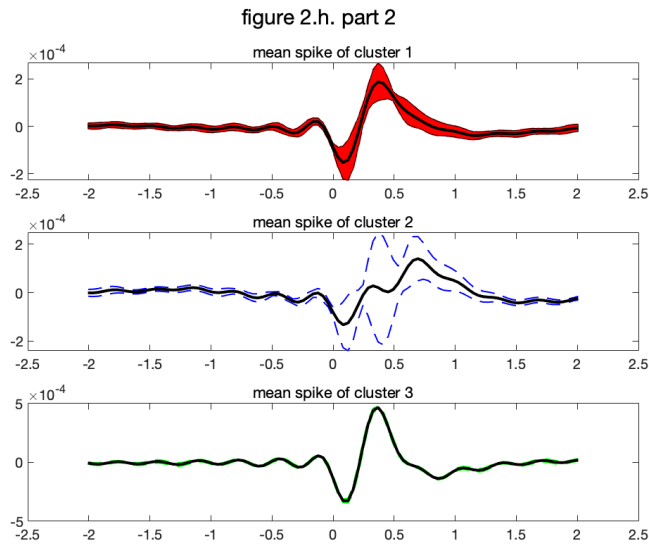
subplot(3,1,2)
plot(tRangeCentered_2, meanSpikeWaveform_2 + 1*std_2, 'k--', 'LineWidth', 1, 'Color', "blue"); %mean + 1sd 그리기
hold on;
plot(tRangeCentered_2, meanSpikeWaveform_2 - 1*std_2, 'k--', 'LineWidth', 1, 'Color', "blue"); %mean - 1sd 그리기
hold on;
plot(tRangeCentered_2, meanSpikeWaveform_2, 'LineWidth', 2, 'Color', 'k'); %meanSpike 그리기
hold off;

title("mean spike of cluster 2")

%cluster 3
[meanSpikeWaveform_3, std_3, tRangeCentered_3, nSpikes_3] = o1_ch8_m1.getClusterMeanSpike(3);

subplot(3,1,3)
plot(tRangeCentered_3, meanSpikeWaveform_3 + 1*std_3, 'k--', 'LineWidth', 1, 'Color', "green"); %mean + 1sd 그리기
hold on;
plot(tRangeCentered_3, meanSpikeWaveform_3 - 1*std_3, 'k--', 'LineWidth', 1, 'Color', "green"); %mean - 1sd 그리기
hold on;
plot(tRangeCentered_3, meanSpikeWaveform_3, 'LineWidth', 2, 'Color', 'k'); %meanSpike 그리기
hold off;
ylim([-2.5, 2.5]*1e4) %ylim 통일 가능
```

```
title("mean spike of cluster 3")
sgtitle("figure 2.h. part 2")
```



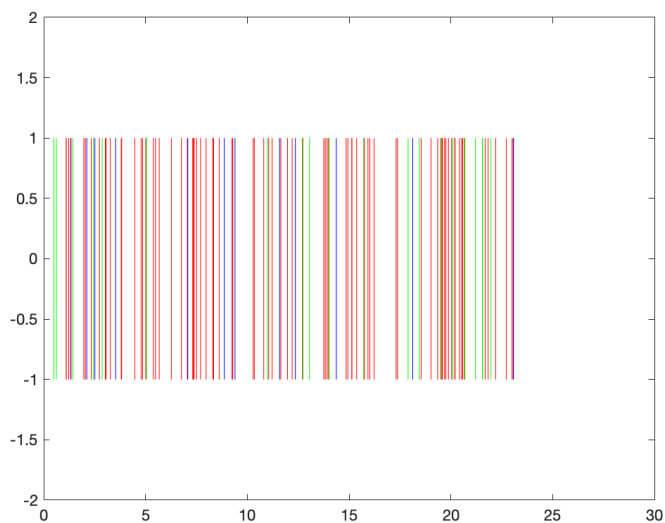
**Figure 2.i**

colored raster plot.

plot 그리는 method:

- `channelobject.drawColoredRaster(colorNamesArray)`: 클러스터별로 색이 다르게 입혀진 raster plot. spike의 위치를 표시.

```
figure
o1_ch8_m1.drawColoredRaster(["red", "blue", "green"]);
xlim([0,30]) %xlim, ylim 조절 가능
```



**Figure 2.j.**

interspike intervals histogram

ISI 데이터 가져오는 method:

- `channelobject.getISValues(clusterNum)`

히스토그램 그리는 matlab 기본 함수 : `histogram`

```
figure
subplot(1,3,1)
[ISValues_1, nSpikes] = o1_ch8_m1.getISValues(1); %cluster 1의 ISI 계산값 가져오기
h1 = histogram(ISValues_1, 20); % bin 개수 20개로 설정. 변경 가능
title('cluster ' + string(1) + ', n = ' + string(nSpikes) ); %subplot 제목
xlabel('ISI (ms)'); % subplot x축 변수 이름
h1.FaceColor = "red"; %색상 지정

subplot(1,3,2)
[ISValues_2, nSpikes_2] = o1_ch8_m1.getISValues(2); %cluster 2의 ISI 계산값 가져오기
```

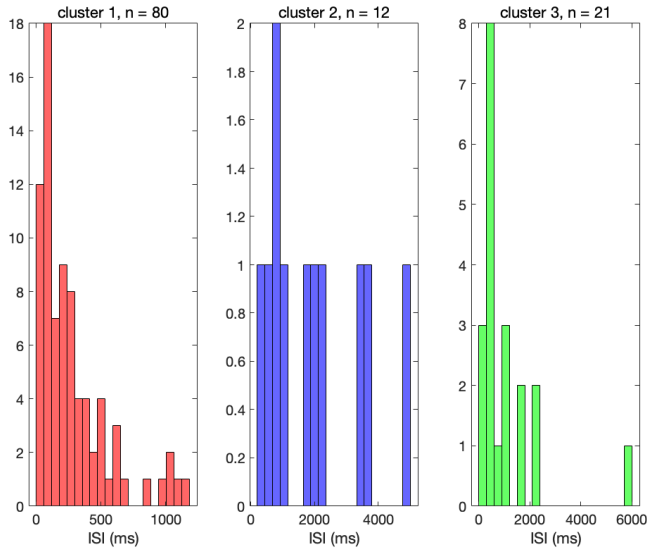


```

h2 = histogram(ISIvalues_2, 20); % bin 개수 20개로 설정. 변경 가능
title('cluster ' + string(2) + ', n = ' + string(nSpikes_2) ); %subplot 제목
xlabel('ISI (ms)'); % subplot x축 변수 이름
h2.FaceColor = "blue"; %색상 지정

subplot(1,3,3)
[ISIvalues_3, nSpikes_3] = o1_ch8_m1.getISIvalues(3); %cluster 3의 ISI 계산값 가져오기
h3 = histogram(ISIvalues_3, 20); % bin 개수 20개로 설정. 변경 가능
title('cluster ' + string(3) + ', n = ' + string(nSpikes_3) ); %subplot 제목
xlabel('ISI (ms)'); % subplot x축 변수 이름
h3.FaceColor = "green"; %색상 지정

```



**Figure 2.k**

theta waves and theta phase

thetaPhase의 경우, y축 범위가  $(-\pi, \pi)$ 임. 논문에서는 y축 스케일을 조절해 trace 모양이 잘 드러나게 하였음. 여기서는 원본 스케일을 써서 논문과 다르게 보임. matlab의 plot함수의 옵션을 사용하여 조절 가능.

channel object에서 데이터 접근 방법

- `channelobject.thetaWaves`: theta wave trace array
- `channelobject.thetaPhases`: theta angle array

```

figure
subplot(3,1,1)
p_raw = plot(o1_ch8_m1.t, o1_ch8_m1.raw); %raw trace plot
p_raw.Color = 'red'; %색깔 조절

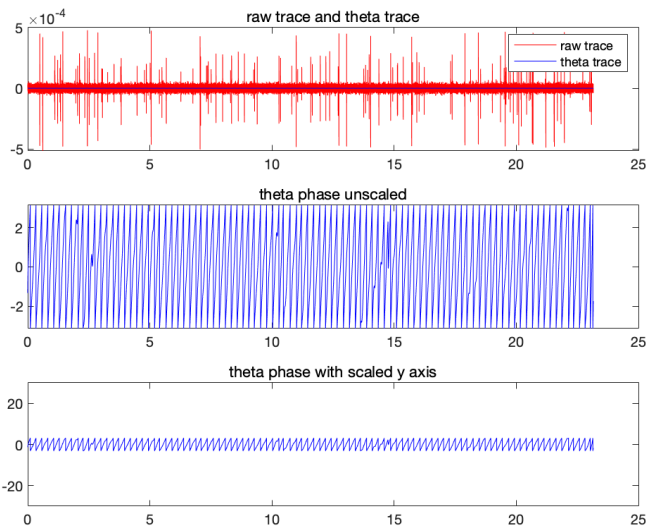
hold on;

p_thetaWaves = plot(o1_ch8_m1.t, o1_ch8_m1.thetaWaves); %theta wave plot
p_thetaWaves.Color = 'blue'; %색깔 조절
legend("raw trace", "theta trace")
title("raw trace and theta trace")
hold off;

subplot(3,1,2)
p_thetaPhases = plot(o1_ch8_m1.t, o1_ch8_m1.thetaPhases);
p_thetaPhases.Color = 'blue'; %색깔 조절
title("theta phase unscaled")

subplot(3,1,3)
p_thetaPhases = plot(o1_ch8_m1.t, o1_ch8_m1.thetaPhases);
p_thetaPhases.Color = 'blue'; %색깔 조절
ylim([-30, 30])
title("theta phase with scaled y axis")

```



**Figure 2.m**

Circular distrubution of theta phases(angles) at spikes

figure 2.j와 비슷한 방식으로 사용

phase 데이터 가져오는 method 이름

- `channelobject.getThetaPhaseByClusterNum(clusterNum)`

circular histogram 그리는 matlab 함수(circstat 애드온): `circ_plot`

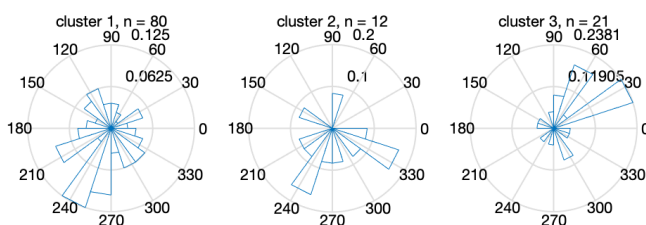
```
figure
subplot(1,3,1)
[phaseValues_1, nSpikes] = o1_ch8_m1.getThetaPhaseByClusterNum(1); %cluster 1의 phase 계산값 가져오기
circ_plot(phaseValues_1,'hist',[],20,true,false); %bin 개수 20개로 설정. 변경 가능
title('cluster ' + string(1) + ', n = ' + string(nSpikes) ); %subplot 제목

subplot(1,3,2)
[phaseValues_2, nSpikes_2] = o1_ch8_m1.getThetaPhaseByClusterNum(2); %cluster 2의 phase 계산값 가져오기
circ_plot(phaseValues_2,'hist',[],20,true,false); %bin 개수 20개로 설정. 변경 가능
title('cluster ' + string(2) + ', n = ' + string(nSpikes_2) ); %subplot 제목

subplot(1,3,3)
[phaseValues_3, nSpikes_3] = o1_ch8_m1.getThetaPhaseByClusterNum(3); %cluster 1의 phase 계산값 가져오기
circ_plot(phaseValues_3,'hist',[],20,true,false); %bin 개수 20개로 설정. 변경 가능
title('cluster ' + string(3) + ', n = ' + string(nSpikes_3) ); %subplot 제목

sgtitle("figure 2.m. circular distrubution of theta phases(angles) at spikes") %전체 제목
```

figure 2.m. circular distrubution of theta phases(angles) at spikes



**Figure 2.n.**

distrubution of theta phases(angles) at spikes

figure 2.m에서 사용한 데이터를 `histgoram` 함수에 적용하면 됨

```

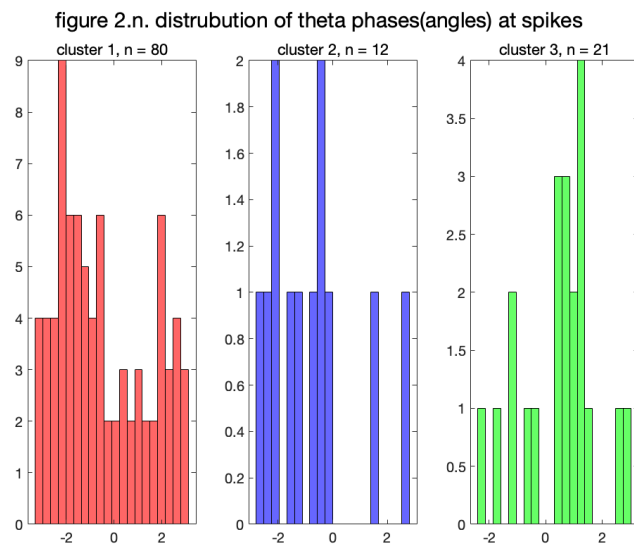
figure
subplot(1,3,1)
[phaseValues_1, nSpikes] = o1_ch8_m1.getThetaPhaseByClusterNum(1); %cluster 1의 phase 계산값 가져오기
h1 = histogram(phaseValues_1, 20); % bin 개수 20개로 설정. 변경 가능
title('cluster ' + string(1) + ', n = ' + string(nSpikes) ); %subplot 제목
h1.FaceColor = "red"; %색상 지정

subplot(1,3,2)
[phaseValues_2, nSpikes_2] = o1_ch8_m1.getThetaPhaseByClusterNum(2); %cluster 2의 phase 계산값 가져오기
h2 = histogram(phaseValues_2, 20); % bin 개수 20개로 설정. 변경 가능
title('cluster ' + string(2) + ', n = ' + string(nSpikes_2) ); %subplot 제목
h2.FaceColor = "blue"; %색상 지정

subplot(1,3,3)
[phaseValues_3, nSpikes_3] = o1_ch8_m1.getThetaPhaseByClusterNum(3); %cluster 1의 phase 계산값 가져오기
h3 = histogram(phaseValues_3, 20); % bin 개수 20개로 설정. 변경 가능
title('cluster ' + string(3) + ', n = ' + string(nSpikes_3) ); %subplot 제목
h3.FaceColor = "green"; %색상 지정

sgtitle("figure 2.n. distrubution of theta phases(angles) at spikes") %전체 제목

```



## Uniform test(phase-locking detection)

testing the non-uniformty of theta phase distribution using Rayleigh criterion to detect phase-locking.

We considered that phase-locking to theta waves happens for P-values below 0.05.

- phase-locking: theta wave의 특정 phase에서 spike가 유독 많이 일어나는 현상. Phase-locking이 없다면, spike에서의 theta angle분포는 uniform 분포를 따를 것. 따라서 theta angle 분포가 uniform distribution과 같은지 가설검정을 수행. 논문에서는 p-value 기준 0.05를 사용. 0.05보다 작으면 phase-locking. 0.05보다 크면 phase-locking 없는 것으로 판단.

test 수행하는 함수 이름

- **channelobject.uniformTest()**: cluster 순서대로 p-를 출력.

```
o1_ch8_m1.uniformTest()
```

```

pvals = 3x1
    0.0175
    0.1159
    0.0160

```

## figure 2 후반부: longtermAnalyzer 클래스 소개 및 plot 재현

월별 비교를 위해 여러 channel의 데이터를 모아서 관리하는 class

일단 여러개의 data file을 불러오고, data 처리를 진행

```
%불러오기
```

```
%organoid, ch, month
```

```

o1_ch1_m1 = file.channel(1, 1, 1);
o1_ch2_m1 = file.channel(1, 2, 1);
o2_ch1_m1 = file.channel(2, 1, 1);
o2_ch2_m2 = file.channel(2, 2, 2);
o3_ch1_m2 = file.channel(3, 1, 2);
o2_ch1_m2 = file.channel(2, 2, 2);

```

```
%bandpass filter
```

```
o1_ch1_m1.bandPass([300,3000]);
o1_ch2_m1.bandPass([300,3000]);
o2_ch1_m1.bandPass([300,3000]);
o2_ch2_m2.bandPass([300,3000]);
o3_ch1_m2.bandPass([300,3000]);
o2_ch1_m2.bandPass([300,3000]);
```

```
%spike detection
```

```
o1_ch1_m1.detectSpikes(3,2,2);
```

```
number of spikes found : 1043
```

```
o1_ch2_m1.detectSpikes(3,2,2);
```

```
number of spikes found : 365
```

```
o2_ch1_m1.detectSpikes(3,2,2);
```

```
number of spikes found : 1043
```

```
o2_ch2_m2.detectSpikes(3,2,2);
```

```
number of spikes found : 365
```

```
o3_ch1_m2.detectSpikes(3,2,2);
```

```
number of spikes found : 1043
```

```
o2_ch1_m2.detectSpikes(3,2,2);
```

```
number of spikes found : 365
```

```
%theta calculation
```

```
o1_ch1_m1.getThetas();
```

```
o1_ch2_m1.getThetas();
```

```
o2_ch1_m1.getThetas();
```

```
o2_ch2_m2.getThetas();
```

```
o3_ch1_m2.getThetas();
```

```
o2_ch1_m2.getThetas();
```

#### longtermAnalyzer object 생성

- **LAobject.addChannel(channelobject)**: 데이터 처리 완료한 channel object를 longtermAnalyzer object에 추가

```
la = longtermAnalyzer(); %longtermAnalyzer object 생성
```

```
%channel object 추가
```

```
la.addChannel(o1_ch1_m1);
```

```
la.addChannel(o1_ch2_m1);
```

```
la.addChannel(o2_ch1_m1);
```

```
la.addChannel(o2_ch2_m2);
```

```
la.addChannel(o3_ch1_m2);
```

```
la.addChannel(o2_ch1_m2);
```

## Figure 2.o

월별 히스토그램.

channel을 지정하면 월별로 모든 organoid를 aggregate.

내부에서 FWHm 계산하는 함수:

- **LAobject.FWHm(spikeWaveform, msPerTs)**: spikewaveform 하나에서 minimum 값의 절반을 계산한 뒤, 그 값보다 작은 값이 나온 첫 번째 timestamp와 마지막 timestamp를 찾아서 그 너비를 milisecond로 환산. Spike가 unimodal이 아닐 경우 FWHm값이 크게 나올 수 있음.
- **LAobject.getFWHmsForChannel**: channel 하나에 있는 모든 spike waveform에 대해 FWHm을 계산

```
figure
```

```
[FWHmPoints_1, organoidLabels_1, meanFWHm_1] = la.getFWHmHistByMonth(1, 1); %month 1, channel 1
```

```
[FWHmPoints_2, organoidLabels_2, meanFWHm_2] = la.getFWHmHistByMonth(2, 1); %month 2, channel 1
```

```
%month 1에는 channel 1이 두 개, month 2에는 channel 1이 한 개 있기 때문에 FWHm 값 개수도
```

```
%channel 1이 channel 2의 두 배임.
```

```
%barplot
```

```
bar([meanFWHm_1, meanFWHm_2]);
```

```
%+- std
```

```
hold on
```

```
%month 1 std line
```

```
monthNow = 1;
```

```
sdFWHm = std(FWHmPoints_1);
```

```
upperLimit = meanFWHm_1 + sdFWHm;
```

```
lowerLimit = meanFWHm_1 - sdFWHm;
```

```

plot([monthNow, monthNow], [lowerLimit,upperLimit], 'k','LineWidth',2); % 세로선
plot([monthNow-0.1, monthNow+0.1], [lowerLimit,lowerLimit], 'k','LineWidth',2); %아래쪽 가로선
plot([monthNow-0.1, monthNow+0.1], [upperLimit,upperLimit], 'k','LineWidth',2); %위쪽 가로선

%%month 2 std line
monthNow = 2;
sdFWHm = std(FWHmPoints_2);
upperLimit = meanFWHm_2 + sdFWHm;
lowerLimit = meanFWHm_2 - sdFWHm;
plot([monthNow, monthNow], [lowerLimit,upperLimit], 'k','LineWidth',2); % 세로선
plot([monthNow-0.1, monthNow+0.1], [lowerLimit,lowerLimit], 'k','LineWidth',2); %아래쪽 가로선
plot([monthNow-0.1, monthNow+0.1], [upperLimit,upperLimit], 'k','LineWidth',2); %위쪽 가로선

%plot points colored by organoids
%% month 1

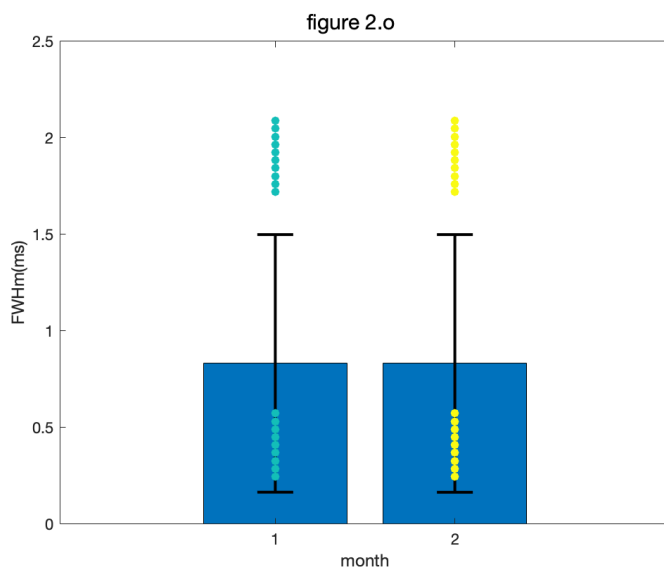
monthNow = 1;
x = repelem(monthNow, length(FWHmPoints_1));
y = FWHmPoints_1;
colors = organoidLabels_1;
scatter(x, y, [], colors, 'filled');

%% month 2
monthNow = 2;
x = repelem(monthNow, length(FWHmPoints_2));
y = FWHmPoints_2;
colors = organoidLabels_2;
scatter(x, y, [], colors, 'filled');

hold off

xlabel("month")
ylabel("FWHm(ms)")
sgtitle("figure 2.o")

```



## Figure 2.p.

phase-space analysis

figure 4.f와 동일

## Figure 2.q.

월별 cluster 개수 비교

월별 mean spike by clusters

월별 colored raster plot

앞의 plot 방법들을 조합하여 해결 가능

## Figure 4.b.

여러 데이터 불러오기

```

                                %organoid, channel, month
o1_ch1_m1 = file.channel(1, 1, 1);
o1_ch2_m1 = file.channel(1, 2, 1);
o1_ch1_m2 = file.channel(1, 3, 2); %month별로 그림이 달라지게 하기 위해 channel 3을 로드
o1_ch2_m2 = file.channel(1, 4, 2); %month별로 그림이 달라지게 하기 위해 channel 4를 로드

```

100-3000hz로 필터링

```
o1_ch1_m1.bandPass([100,3000])
o1_ch2_m1.bandPass([100,3000])
o1_ch1_m2.bandPass([100,3000])
o1_ch2_m2.bandPass([100,3000])
```

cluster 개수 3개, spike waveform 길이 +- 2ms로 spike detection 실행

```
o1_ch1_m1.detectSpikes(3,2,2)
```

number of spikes found : 1407

```
o1_ch2_m1.detectSpikes(3,2,2)
```

number of spikes found : 334

```
o1_ch1_m2.detectSpikes(3,2,2)
```

number of spikes found : 259

```
o1_ch2_m2.detectSpikes(3,2,2)
```

number of spikes found : 312

channel object별로 raw trace 그리기. ylim 옵션을 이용해 비교대상끼리 y축 스케일 통일.

```
figure
%month 1

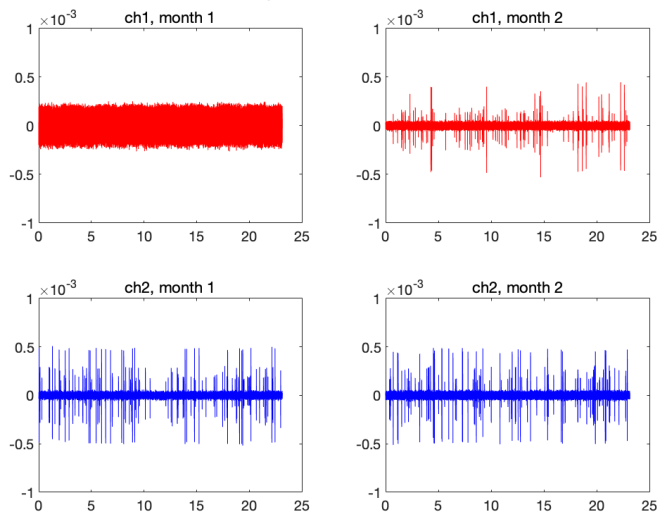
%ch1
subplot(2,2,1)
p = plot(o1_ch1_m1.t, o1_ch1_m1.raw);
p.Color = 'red';
title('ch1, month 1');
ylim([-1,1]*1e-3);%ylim 조절

%ch2
subplot(2,2,3)
p_o1_ch2_m1 = plot(o1_ch2_m1.t, o1_ch2_m1.raw);
p_o1_ch2_m1.Color = 'blue';
title('ch2, month 1');
ylim([-1,1]*1e-3);%ylim 조절

%month 2
%ch1
subplot(2,2,2)
p_o1_ch1_m2 = plot(o1_ch1_m2.t, o1_ch1_m2.raw);
p_o1_ch1_m2.Color = 'red';
title('ch1, month 2');
ylim([-1,1]*1e-3);%ylim 조절

%ch2
subplot(2,2,4)
p_o1_ch2_m2 = plot(o1_ch2_m2.t, o1_ch2_m2.raw);
p_o1_ch2_m2.Color = 'blue';
title('ch2, month 2')
ylim([-1,1]*1e-3);%ylim 조절
%
sgtitle("figure 4.b. part1.")
```

figure 4.b. part1.



channel object별로 mean spike 그리기.

channel object의 전체 mean spike 정보 가져오는 method:

- `channelobject.getTotalMeanSpike()`

figure

```
[meanSpikeWaveform_1, std_1, tRangeCentered_1, nSpikes_1] = o1_ch1_m1.getTotalMeanSpike();
[meanSpikeWaveform_2, std_2, tRangeCentered_2, nSpikes_2] = o1_ch2_m1.getTotalMeanSpike();
[meanSpikeWaveform_3, std_3, tRangeCentered_3, nSpikes_3] = o1_ch1_m2.getTotalMeanSpike();
[meanSpikeWaveform_4, std_4, tRangeCentered_4, nSpikes_4] = o1_ch2_m2.getTotalMeanSpike();

subplot(4,1,1)
plot(tRangeCentered_1, meanSpikeWaveform_1 + 1*std_1, 'k--', 'LineWidth', 1, 'Color', "red"); %mean + 1sd 그리기
hold on;
plot(tRangeCentered_1, meanSpikeWaveform_1 - 1*std_1, 'k--', 'LineWidth', 1, 'Color', "red"); %mean - 1sd 그리기
hold on;

%curve 사이 색칠하기(오류 발생 가능)
t2 = [tRangeCentered_1, fliplr(tRangeCentered_1)];
inBetween = [meanSpikeWaveform_1 - 1*std_1, fliplr(meanSpikeWaveform_1 + 1*std_1)];
fill(t2, inBetween, "red");
hold on;

plot(tRangeCentered_1, meanSpikeWaveform_1, 'LineWidth', 2, 'Color', 'k'); %meanSpike 그리기

hold off;

title('mean spike of ch1, month1');

subplot(4,1,2)
plot(tRangeCentered_2, meanSpikeWaveform_2 + 1*std_2, 'k--', 'LineWidth', 1, 'Color', "blue"); %mean + 1sd 그리기
hold on;
plot(tRangeCentered_2, meanSpikeWaveform_2 - 1*std_2, 'k--', 'LineWidth', 1, 'Color', "blue"); %mean - 1sd 그리기
hold on;
plot(tRangeCentered_2, meanSpikeWaveform_2, 'LineWidth', 2, 'Color', 'k'); %meanSpike 그리기
hold off;
title('mean spike of ch2, month1');

subplot(4,1,3)
plot(tRangeCentered_3, meanSpikeWaveform_3 + 1*std_3, 'k--', 'LineWidth', 1, 'Color', "green"); %mean + 1sd 그리기
hold on;
plot(tRangeCentered_3, meanSpikeWaveform_3 - 1*std_3, 'k--', 'LineWidth', 1, 'Color', "green"); %mean - 1sd 그리기
hold on;
plot(tRangeCentered_3, meanSpikeWaveform_3, 'LineWidth', 2, 'Color', 'k'); %meanSpike 그리기
hold off;
title('mean spike of ch1, month2');

subplot(4,1,4)
plot(tRangeCentered_4, meanSpikeWaveform_4 + 1*std_4, 'k--', 'LineWidth', 1, 'Color', "magenta"); %mean + 1sd 그리기
hold on;
plot(tRangeCentered_4, meanSpikeWaveform_4 - 1*std_4, 'k--', 'LineWidth', 1, 'Color', "magenta"); %mean - 1sd 그리기
hold on;
```

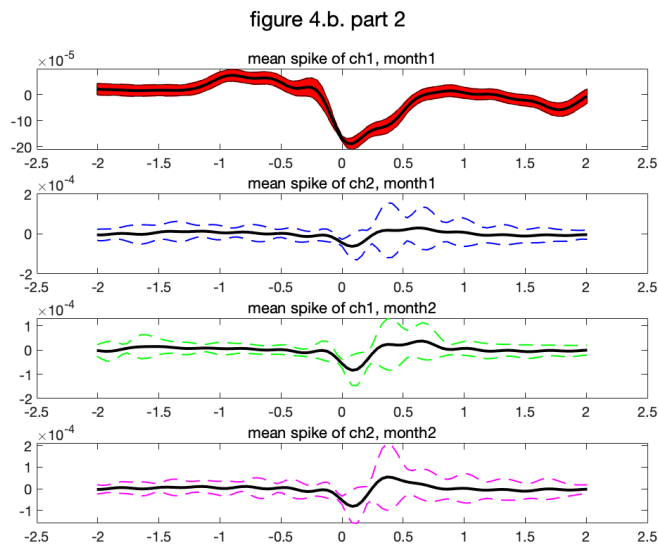
```

plot(tRangeCentered_4, meanSpikeWaveform_4, 'Linewidth',2,'Color','k'); %meanSpike 그리기
hold off;

title('mean spike of ch2, month2');

sgtitle("figure 4.b. part 2")

```



**Figure 4.c.**

spectrogram

x축 0s = 데이터 시작 시간(이 데이터에서는 약 30s)

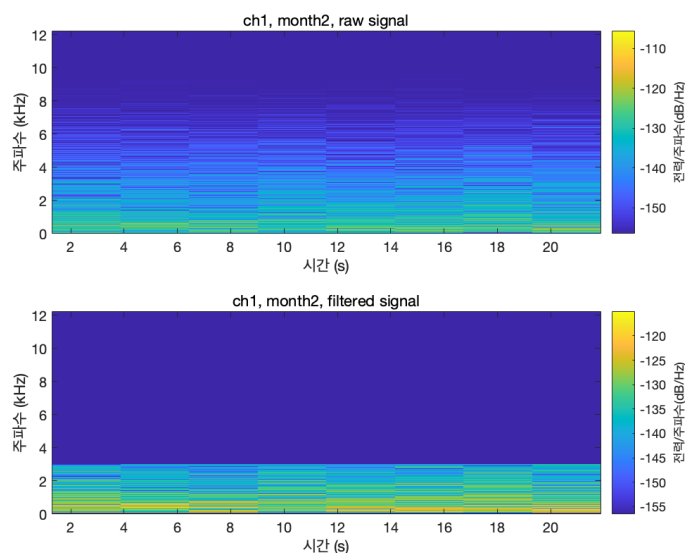
matlab signal processing toolbox의 spectrogram 함수 사용

```

figure
subplot(2,1,1)
spectrogram(o1_ch1_m2.raw, [], [], [], o1_ch1_m2.sf, 'yaxis')
title('ch1, month2, raw signal')%raw

subplot(2,1,2)
spectrogram(o1_ch1_m2.filtered, [], [], [], o1_ch1_m2.sf, 'yaxis')
title('ch1, month2, filtered signal')

```



**Figure 4.d**

power spectrums

signal processing toolbox의 pspectrum 함수 사용

```

figure

xTable = timetable(seconds(o1_ch1_m1.t), o1_ch1_m1.raw);
[pxx,f] = pspectrum(xTable);
month1 = plot(f,pow2db(pxx));
month1.Color = 'red';

```



```

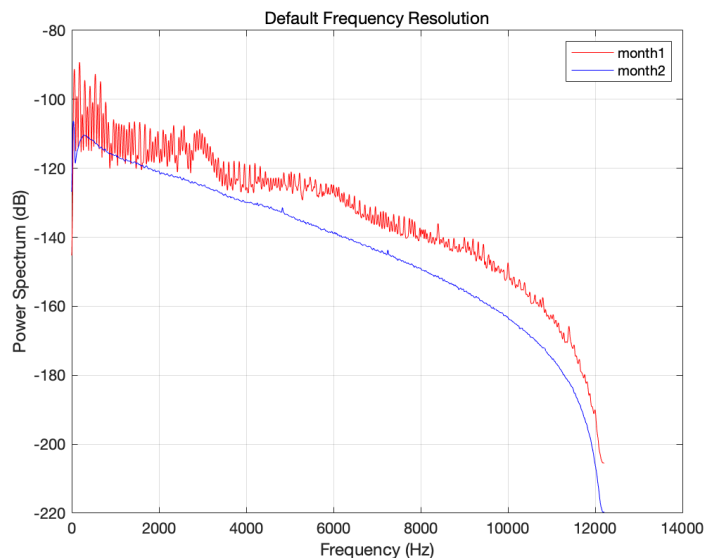
hold on

xTable = timetable(seconds(o1_ch1_m2.t), o1_ch1_m2.raw);
[pxx,f] = pspectrum(xTable);
month2 = plot(f,pow2db(pxx));
month2.Color = 'blue';

grid on

legend('month1', 'month2');
xlabel('Frequency (Hz)');
ylabel('Power Spectrum (dB)');
title('Default Frequency Resolution');

```



**Figure 4.f.**

phase-space analysis

plot 그리는 함수 이름

- **channelobject.drawPhaseSpace(color)**: channel의 dVdt - V trajectory를 그림. 색상 지정 가능.

월별로 mean spike를 비교하였으나, 어떤 waveform이라도 대입하면 그릴 수 있음

"normalize" : v, dv/dt를 각각의 minimum으로 나눔

```

figure

%month 1
averageWaveform = mean(o1_ch1_m1.spikeWaveforms);
dVdt = diff(averageWaveform)/o1_ch1_m1.msPerTs;

%%normalize
averageWaveform = averageWaveform / min(averageWaveform);
dVdt = dVdt / min(dVdt);

%algorithm for drawing pahse space plot
color = "red";
V = averageWaveform(1:(end-1));
hold on
for i = 1: (length(dVdt)-1)
    p = plot([V(i), V(i+1)], [dVdt(i), dVdt(i+1)], 'k');
    p.Color = color;
end

%month 2
averageWaveform = mean(o1_ch1_m2.spikeWaveforms);
dVdt = diff(averageWaveform)/o1_ch1_m2.msPerTs;

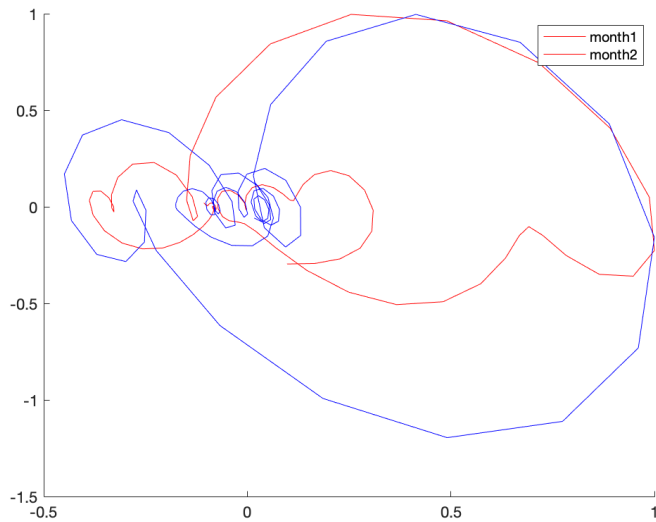
%normalize
averageWaveform = averageWaveform / min(averageWaveform);
dVdt = dVdt / min(dVdt);

```

```

%algorithm for drawing pahse space plot
color = "blue";
V = averageWaveform(1:(end-1));
hold on
for i = 1: (length(dVdt)-1)
    p = plot([V(i), V(i+1)], [dVdt(i), dVdt(i+1)], 'k');
    p.Color = color;
end
hold off
legend('month1', 'month2')

```



- figure 4.g.는 figure 2.o와 동일
- figure 4.e, h는 figure 2.o에서 FWHm만 signal power와 spikes count per neuron으로 바꾸면 됨
- figure 4.i 아직 이해 못함
- figure 5는 figure 2의 방법으로 모두 구현 가능

o1\_ch1\_m1

o1\_ch1\_m1 =

channel\_from\_data - 속성 있음:

```

organoidNum: 1
channelNum: 1
month: 1
nObsPerRow: 2048
sf: 2.4414e+04
msPerTs: 0.0410
nTimestamps: 565248
startTime: 0
t: [565248x1 double]
raw: [565248x1 double]
filtered: [565248x1 double]
timestampsPrePeak: 49

```